ReBAC V1 — Basic Relationship-Based Access Control

> ***Branch:*** `ReBAC-V1` | ***Purpose:*** *Establish the foundational graph model*

The Problem This Version Solves

Traditional Role-Based Access Control (RBAC) assigns permissions to roles and roles to users. This breaks down in healthcare, multi-tenant SaaS, and any domain where **who can access what depends on the relationship between the subject and the object**, not just the subject's role.

Example failure of RBAC:

```
ROLE: doctor
CAN: view patients
```

This grants Dr. Raj access to **all** patients — not just his own. That is a security violation.

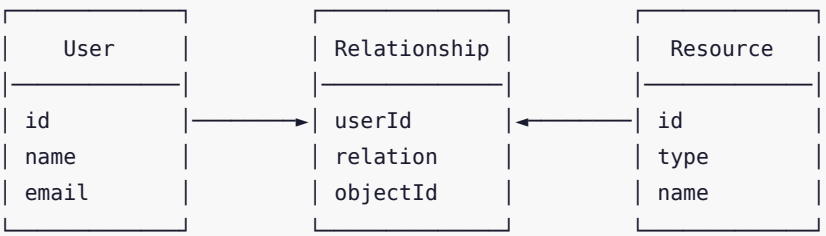
What we need:

```
Raj ——[doctor_of]——> Patient101   ✔ Allowed
Raj ——[doctor_of]——> Patient202?  ✘ No relationship → Denied
```

The permission is encoded in the **edge** of a directed graph, not in a role table.

Version 1 Architecture

Core Data Model



Permission Check Algorithm

```
checkPermission(userId, resourceId, permission):
  1. Look up permission rule from schema
  2. If rule.relation == "doctor_of":
       SELECT 1 FROM Relationship
       WHERE userId = ? AND objectId = ? AND relation = "doctor_of"
  3. Return found/not-found
```

Complexity: O(1) — single indexed lookup.

Technology Choices in V1

Component	Choice	Rationale
Database	PostgreSQL	ACID transactions, foreign keys
ORM	Prisma	Type-safe queries, migrations
Runtime	Node.js + TypeScript	Developer velocity
Authorization layer	Custom graph engine	Learning & full control

---

Critical Review

✅ What V1 Gets Right

1. **Graph model is the correct primitive.** Google Zanzibar, SpiceDB, and OpenFGA all model authorization as directed graphs of tuples `(subject, relation, object)`.

2. **No role-table — correct.** Direct relationship encoding means per-resource, per-user specificity.
3. **Separation of data and policy** — relationships stored in DB, rules defined separately.

⚠️ What V1 Is Missing

Missing: Wildcard / public access `(*) —[viewer]—> document` — no way to express "anyone can view".

Missing: Subject set (group membership) Only individual users can hold relationships. Groups, teams, departments are not modeled.

Missing: Hierarchical permissions A nurse assigned to a ward cannot automatically access patients *inside* that ward.

Missing: Negative permissions / DENY rules No way to explicitly block a relationship.

❌ Incorrect Assumptions in V1

1. **Flat schema** — all relationships are stored in one table with a `relation` string. This is correct for Zanzibar but V1 lacks any type system — `doctor_of` on a `Department` resource is accepted silently.
2. **Permission = relation name** — V1 conflates relations and permissions. In Zanzibar, these are **distinct**: a *relation* is a direct edge; a *permission* is a computed set derived from relations.
3. **No zookie / consistency token** — Zanzibar uses *zookies* (consistency tokens) to guarantee freshness guarantees. V1 will read stale data under concurrent writes.

Comparison with Industry Standards

Concept	V1	Google Zanzibar	SpiceDB	OpenFGA
Tuple model	✅ `(user, relation, resource)`	✅ `(user#rel, relation, object#rel)`	✅ full	✅ full
Subject sets	❌	✅	✅	✅
Typed relations	❌	✅	✅	✅
Consistency tokens	❌	✅ zookies	✅ ZedTokens	✅

Recursive permissions	❌	✅	✅	✅

What The Next Version Must Fix

1. **Recursive permission evaluation** — `ward.contains → patient.view` chain
2. **Subject abstraction** — groups, teams, departments
3. **Typed relations** — `relation doctor_of : user`

Summary

V1 correctly establishes the **tuple-based directed graph** as the fundamental authorization primitive, mirroring the design of Google Zanzibar. It is a correct minimal starting point but lacks recursive evaluation, typed relations, and group membership — all of which are required for any real-world deployment.